

Chapter 6

Arithmetic Unit

Contents:

1. Representation of Binary number and Arithmetic in Unsigned Notation
2. Addition and Subtraction in Unsigned Notation
3. Multiplication in Unsigned Notation, Shift Add Multiplication Algorithm, Booth's Algorithm
4. Division in Unsigned Notation, Shift Subtract Division Algorithm
5. Signed Notation
6. Addition and Subtraction in Signed Notation
7. Binary Coded Decimal (BCD), BCD Numeric format, BCD Addition
8. Specialized Arithmetic Hardware: Lookup ROM, Wallace Tree, Arithmetic Pipeline
9. Floating Point Numbers, Numeric Format
10. IEEE 754 Floating Point Standard, numeric Format

6.1 Arithmetic Unit

- The Arithmetic unit is a part of processor that **executes arithmetic operations**.
- A **microprocessor is capable of processing numeric data in one or more formats**.

Data Representation

- Data representation should require three provisions:
 - Provision for +ve number(or Unsigned number) representation
 - Provision for –ve number (or signed number) representation.
 - Provision for fractional number representation.

Representation of Binary number and Arithmetic in signed Notation

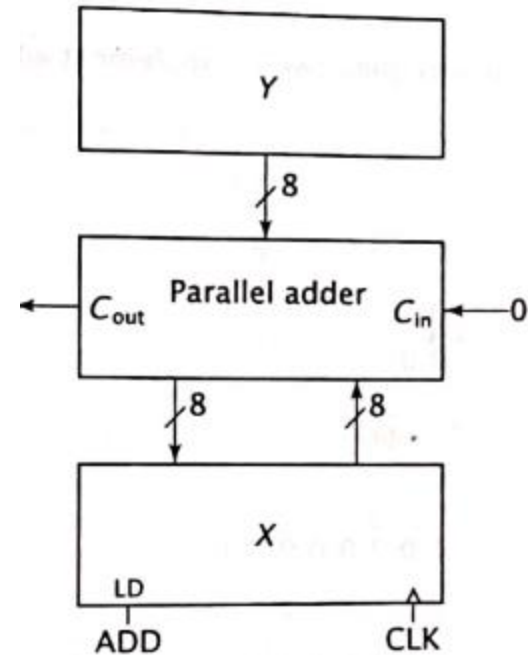
a) Unsigned Number Representation

- In **unsigned** notation, there is **no any separate bit** to represent the sign (Positive or Negative) of the number.
- Eg:
 - 1111
 - This is equivalent to 15 in decimal value.

Binary Representation	Unsigned Non-Negative	Unsigned Two's Complement
0000 0000	0	0
0000 0001	1	1
...	...	...
0111 1111	127	127
1000 0000	128	-128
1000 0001	129	-127
...	...	...
1111 1111	255	-1

i) Addition in Unsigned Notation

- For both the non-negative and two's complement notations, addition and subtraction are fairly straightforward.
- Addition is implemented as a straight binary addition.
- It is realized in hardware by using a **parallel adder** as shown in the figure.
- Here, X and Y are 8-bit registers.
- The circuit performs the micro-operation
- **ADD: $X \leftarrow X + Y$.**



Implementation of the micro-operation $X \leftarrow X + Y$

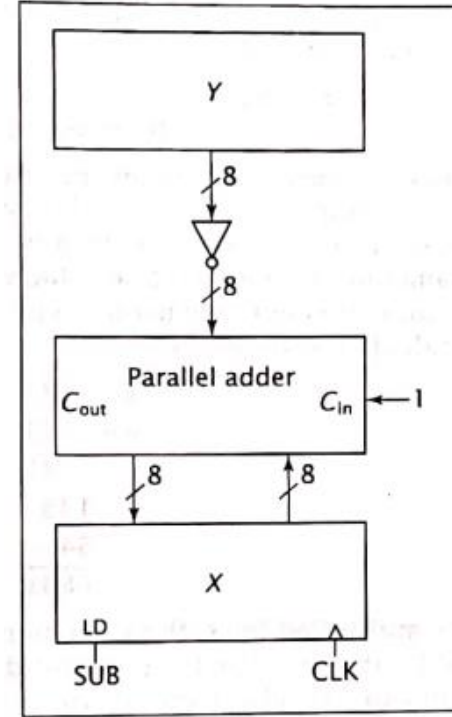
Overflow during Addition in Unsigned Notation

- **Overflow** only occurs when two numbers with the **same sign** are added.
- Adding two numbers with **different signs always produce a valid result**.
- Subtraction can be converted to addition and performed in a similar manner.

126	0 1 1 1 1 1 1 0	127	0 1 1 1 1 1 1 1
+1	<u>0 0 0 0 0 0 0 1</u>	+1	<u>0 0 0 0 0 0 0 1</u>
	0 1 1 1 1 1 1 1		1 0 0 0 0 0 0 0
	0 0		0 1
	(a)		(b)
-127	1 0 0 0 0 0 0 1	-128	1 0 0 0 0 0 0 0
+(-1)	<u>1 1 1 1 1 1 1 1</u>	+(-1)	<u>1 1 1 1 1 1 1 1</u>
	1 0 0 0 0 0 0 0		0 1 1 1 1 1 1 1
	1 1		1 0
	(c)		(d)

ii) Subtraction in Unsigned Notation

- Subtracting numbers in two's complement notation is also implemented as $X - Y = X + (-Y)$.



Implementation of the micro-operation $X \leftarrow X - Y$

Overflow during Subtraction in Unsigned Notation

2	00000010	1	00000001
-1	<u>11111111</u>	-2	<u>11111110</u>
	100000001		011111111
(a)		(b)	
255	11111111	254	11111110
-254	<u>00000010</u>	-255	<u>00000001</u>
	100000001		011111111
(c)		(d)	

Overflow generation in unsigned two's complement subtraction

b) Signed Number Representation

- There are three ways to represent –ve numbers.
 - Signed Magnitude Representation
 - 1's Complement Representation
 - 2's Complement Representation

1. ***Signed Magnitude Representation***

- Here, an extra bit is required as MSB to represent the sign of a number.
- If sign bit is 0, the number is +ve.
- If sign bit is 1, the number is -ve.
- A n bit number requires n+1 bits.

Eg: +10 = 01010

 -10 = 11010

Limitation:

- It represents +0 and -0 by different value.

Eg: +0 = 00000

 - 0 = 10000

- Signed bit must be considered while performing addition or subtraction.

2) *One's Complement Representation*

- Here, negative numbers are represented by complementing each bit.
- Eg: $+6 = 0110$
 $-6 = 1001$
- Limitation: It represents +0 and -0 by different value.
- Eg:
 - $+0 = 0000$
 - $-0 = 1111$

3) Two's Complement Representation

- Two's complement representation overcomes the fore-mentioned limitation.
- Two's complement = One's complement + 1
- Eg:
 - $+6 = 0110$
 - $-6 = 2's \text{ complement of } 0110 = 1001 + 1 = 1010$

b) Fractional Number Representation

i) Fixed Point Representation

The decimal position is either at beginning or at end of the number.

$$\begin{aligned}\text{Eg: } 2.2 &= 0.22 \times 10 \\ &= 22 \times 10^{-1}\end{aligned}$$

ii) Floating Point Representation

Here, a second register is separately used to determine the position of decimal.

Addition and Subtraction in signed Notation

6.3 Multiplication Algorithm

- Unsigned Multiplication
 - Shift-Add Algorithm
- Signed Multiplication
 - Booth's Algorithm

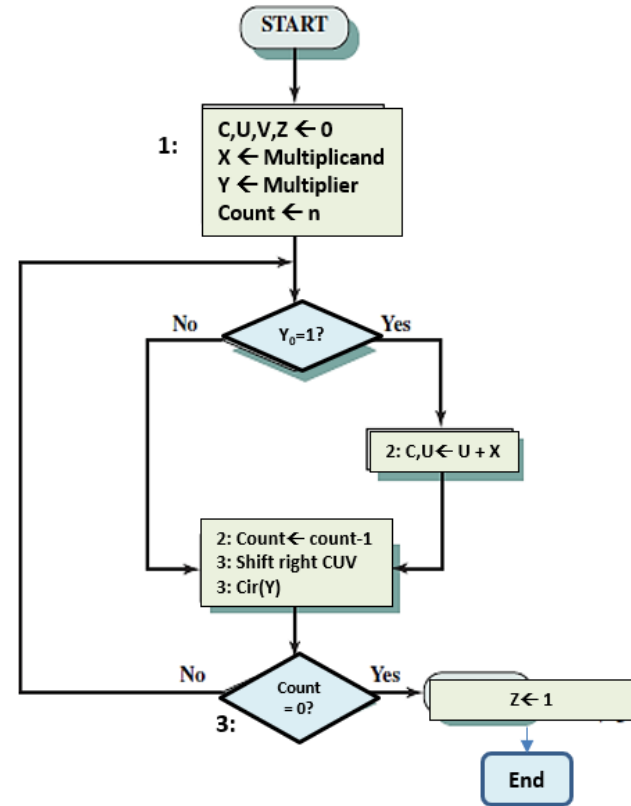
a) Shift Add Algorithm for Unsigned Multiplication

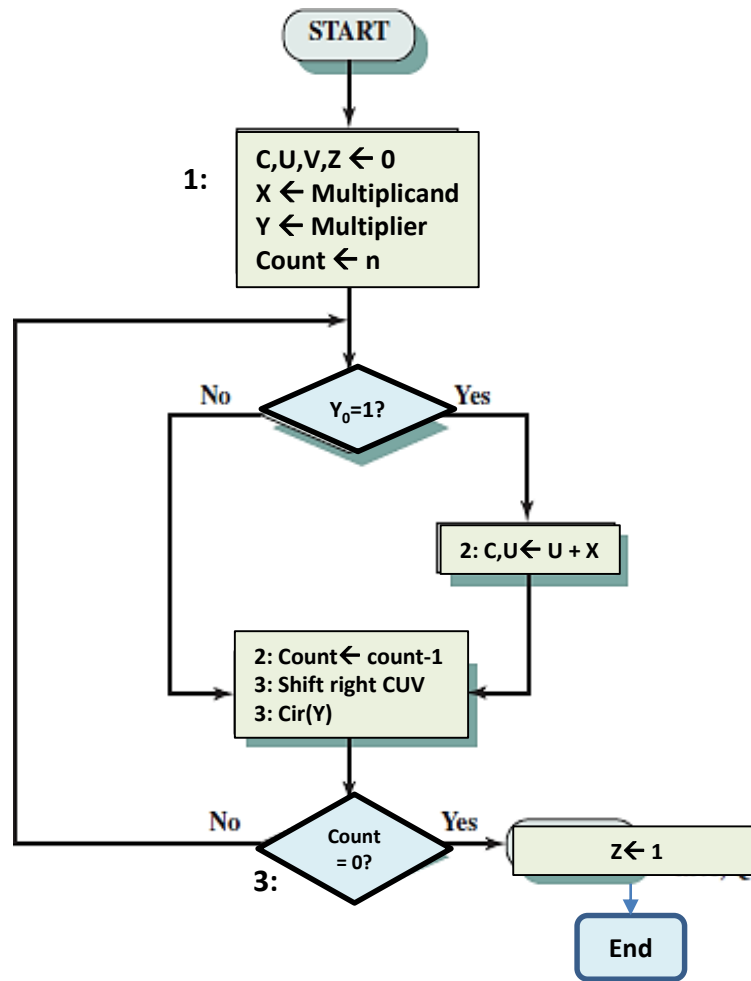
- Algorithm

```
U ← 0;  
FOR i = 1 TO n DO  
  {IF  $Y_0 = 1$  THEN  $CU = U + X$ ;  
  linear shift right CUV;  
  circular shift right Y}
```

- RTL Code

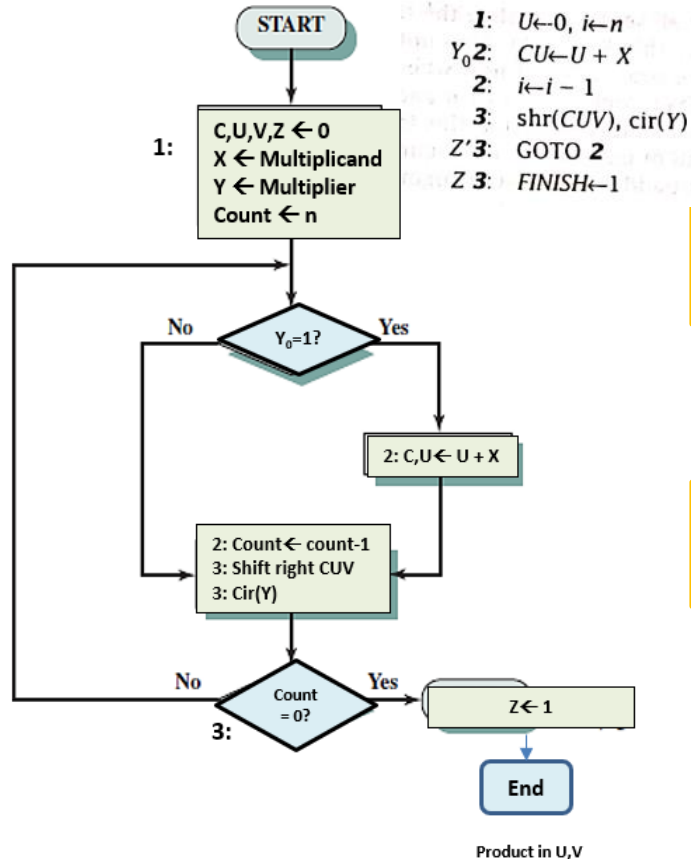
```
1:  $U \leftarrow 0, i \leftarrow n$   
 $Y_0$  2:  $CU \leftarrow U + X$   
2:  $i \leftarrow i - 1$   
3:  $\text{shr}(CUV), \text{cir}(Y)$   
 $Z'$  3: GOTO 2  
 $Z$  3:  $\text{FINISH} \leftarrow 1$ 
```





Product in U,V

RTL code for Unsigned Multiplication for 13 x11



Conditions	Micro-operations	i	C	U	V	Y	Z	FINISH
START		x	x	xxxx	xxxx	1011		0
1	$U \leftarrow 0, i \leftarrow 4$	4		0000			0	
Y_0 2,2	$CU \leftarrow U + X, i \leftarrow i - 1$	3	0	1101			0	
3,Z'3	$\text{shr}(CUV), \text{cir}(Y), \text{GOTO } 2$		0	0110	1xxx	1101		
Y_0 2,2	$CU \leftarrow U + X, i \leftarrow i - 1$	2	1	0011			0	
3,Z'3	$\text{shr}(CUV), \text{cir}(Y), \text{GOTO } 2$		0	1001	11xx	1110		
2	$i \leftarrow i - 1$	1					0	
3,Z'3	$\text{shr}(CUV), \text{cir}(Y), \text{GOTO } 2$		0	0100	111x	0111		
Y_0 2,2	$CU \leftarrow U + X, i \leftarrow i - 1$	0	1	0001			1	
3,Z3	$\text{shr}(CUV), \text{cir}(Y), \text{FINISH} \leftarrow 1$		0	1000	1111	1011		1

Trace of the RTL code for the shift-add algorithm

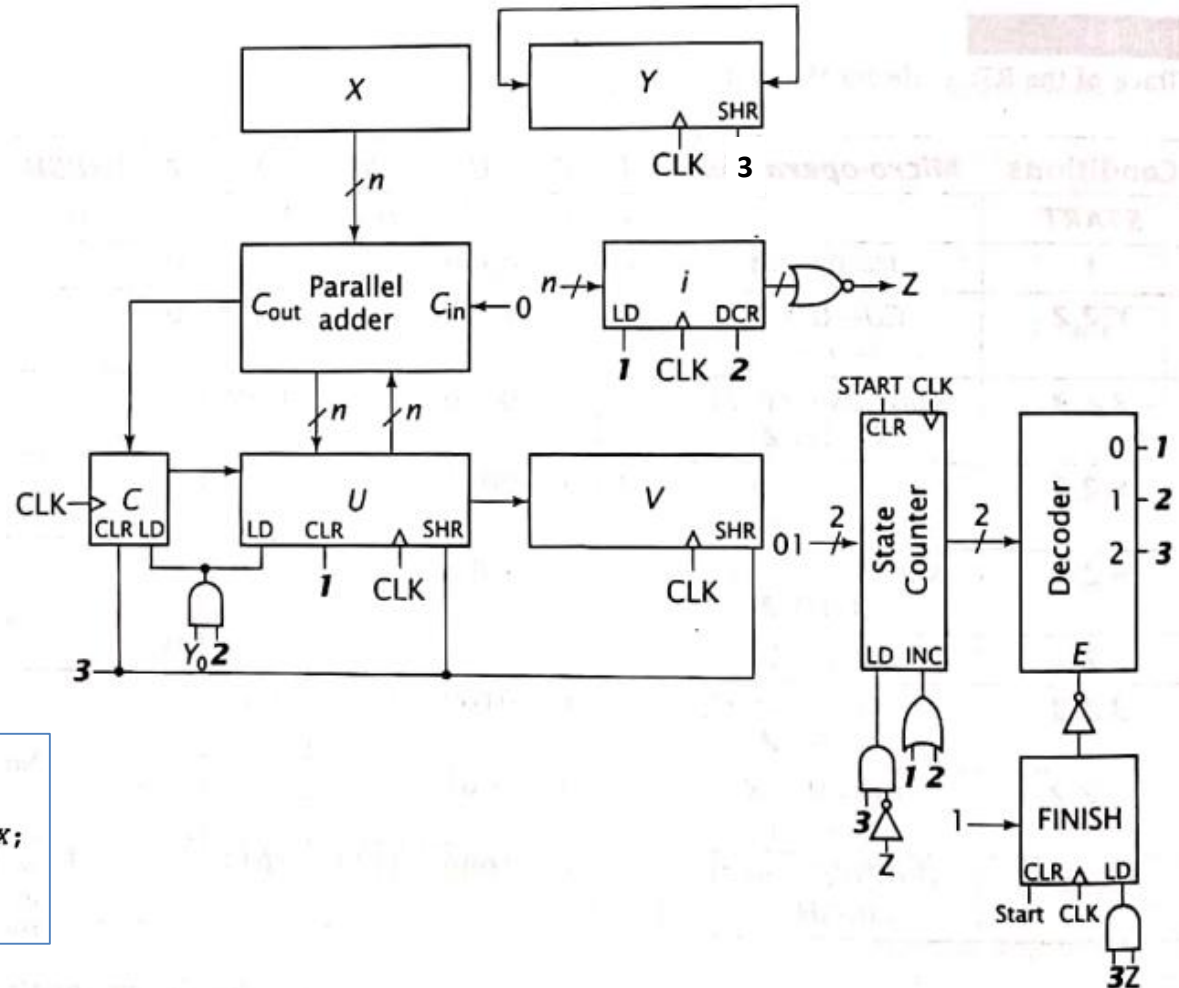
Hardware Implementation of Shift-Add Multiplication Algorithm

```

1:  $U \leftarrow 0, i \leftarrow n$ 
 $Y_0$  2:  $CU \leftarrow U + X$ 
2:  $i \leftarrow i - 1$ 
3: shr(CUV), cir(Y)
Z' 3: GOTO 2
Z 3: FINISH  $\leftarrow 1$ 
    
```

```

U = 0;
FOR i = 1 TO n DO
    {IF  $Y_0 = 1$  THEN  $CU = U + X$ ;
    linear shift right CUV;
    circular shift right Y}
    
```



Classwork:

1. Perform: $10_{10} \times 12_{10} = ?$

b) Signed Multiplication

- It is also called Booth's algorithm or Two's complement multiplication.
- Here, negative numbers are represented by 2's complement representation.

b) Booth's Algorithm for Signed Multiplication

• Algorithm

```

U = 0; Y-1 = 0;
FOR i = 1 TO n DO
{IF start of a string of 1's in Y THEN U = U - X (= U + X' + 1);
 IF end of a string of 1's in Y THEN U = U + X;
 arithmetic shift right UV;
 circular shift right Y AND copy Y0 to Y-1}

```



• RTL Code

```

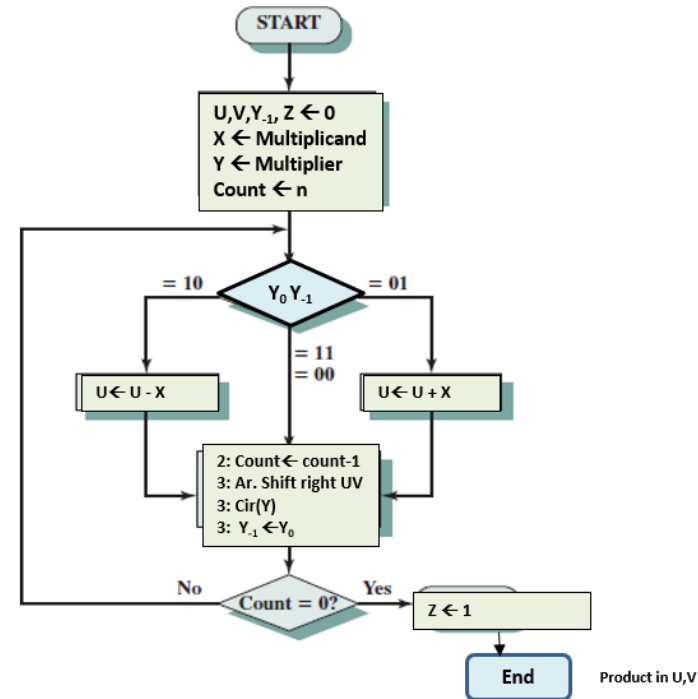
U=0; Y-1=0;
FOR i=1 to n DO
{
IF Y0Y-1= 01, THEN U=U+X
ELSE IF Y0Y-1 = 10, THEN U=U+X
Arithmetic Shift Right CUV;
Circular Right Shift Y;
Y-1←Y0;
}

```

```

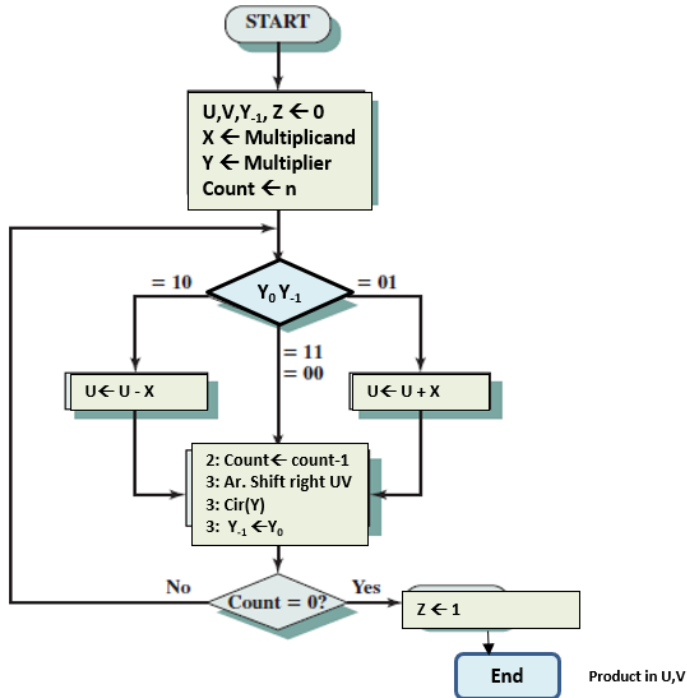
1: U←0, Y-1←0, i←n
Y0Y-1: 2: U←U + X' + 1
Y0'Y-1: 2: U←U + X
2: i←i - 1
3: ashr(UV), cir(Y), Y-1←Y0
Z'3: GOTO 2
Z3: FINISH←1

```



RTL code for Booth's Algorithm for -3 x -5

1: $U \leftarrow 0, Y_{-1} \leftarrow 0, i \leftarrow n$
2: $Y_0 Y_{-1} \text{ '2,2: } U \leftarrow U + X' + 1$
3: $Y_0 Y_{-1} \text{ '2: } U \leftarrow U + X$
2: $i \leftarrow i - 1$
3: $\text{ashr}(UV), \text{cir}(Y), Y_{-1} \leftarrow Y_0$
Z'3: **GOTO 2**
Z3: **FINISH** $\leftarrow 1$



Conditions	Micro-operations	i	U	V	Y	Y ₋₁	Z	FINISH
START		x	xxxx	xxxx	1011	x		0
1	$U \leftarrow 0, Y_{-1} \leftarrow 0, i \leftarrow 4$	4	0000			0	0	
$Y_0 Y_{-1} \text{ '2,2}$	$U \leftarrow U + X' + 1, i \leftarrow i - 1$	3	0011				0	
3,Z'3	$\text{ashr}(UV), \text{cir}(Y), Y_{-1} \leftarrow Y_0, \text{GOTO 2}$		0001	1xxx	1101	1		
2	$i \leftarrow i - 1$	2					0	
3,Z'3	$\text{ashr}(UV), \text{cir}(Y), Y_{-1} \leftarrow Y_0, \text{GOTO 2}$		0000	11xx	1110	1		
$Y_0 Y_{-1} \text{ '2,2}$	$U \leftarrow U + X, i \leftarrow i - 1$	1	1101				0	
3,Z'3	$\text{ashr}(UV), \text{cir}(Y), Y_{-1} \leftarrow Y_0, \text{GOTO 2}$		1110	111x	0111	0		
$Y_0 Y_{-1} \text{ '2,2}$	$U \leftarrow U + X' + 1, i \leftarrow i - 1$	0	0001				1	
3,Z3	$\text{ashr}(UV), \text{cir}(Y), Y_{-1} \leftarrow Y_0, \text{FINISH} \leftarrow 1$		0000	1111	1011			1

Note:

If you get 1 at MSB of U, then your result is negative. So, you need to re-complement it to verify your answer.

Trace of the RTL code for Booth's algorithm

Reminder Note:

- If you are are given -13 and -4 to be multiplied.
- First find 2's complement of each negative numbers.
- Eg: -13 = 2's complement of 1101 = $0010 + 1 = 0011$
- Then take one more extra bit for sign.
- ie. -13 = 10011
- Similarly, -4 = 2's complement of 00100 = $11011 + 1 = 11100$
- However, for multiplication of -7 and -5, only 3 bits are enough to represent each of them. So, we can be take only 4 bits for each.
- Alert! If you do not take 1 extra bit, your calculation will be considered totally wrong.

Hardware Implementation of Booth's Algorithm

```

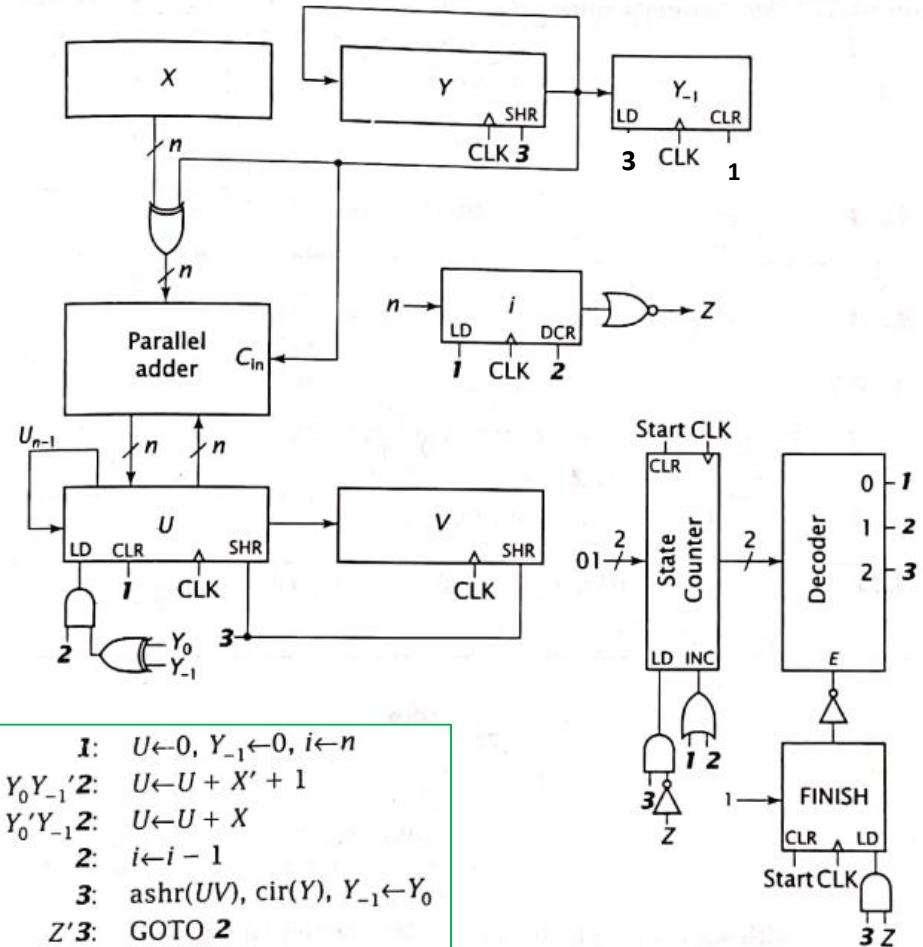
U = 0; Y-1 = 0;
FOR i = 1 TO n DO
{IF start of a string of 1's in Y THEN U = U - X (= U + X' + 1);
 IF end of a string of 1's in Y THEN U = U + X;
 arithmetic shift right UV;
 circular shift right Y AND copy Y0 to Y-1}

```

```

1: U ← 0, Y-1 ← 0, i ← n
Y0Y-1'2: U ← U + X' + 1
Y0'Y-12: U ← U + X
2: i ← i - 1
3: ashr(UV), cir(Y), Y-1 ← Y0
Z'3: GOTO 2
Z3: FINISH ← 1

```



Classwork:

1. Multiply 4×-8 using Booth's Algorithm.
2. Perform: $7_{10} \times -3_{10} = ?$
3. Perform: $9_{10} \times -3_{10} = ?$

[Hint: Take 5 bits for both X and Y]

6.4 Division Algorithm

- Division in Unsigned notation
 - Shift-Subtract Division Algorithm

- Algorithm

```
IF  $U \geq X$  THEN exit with overflow;  
 $Y = 0$ ;  $C = 0$ ;  
FOR  $i = 1$  TO  $n$  DO  
    {linear shift left  $CUV$ ;  
    linear shift left  $Y$ ;  
    IF  $CU \geq X$  THEN  $\{Y_0 = 1, U = CU - X\}$ }
```

- RTL code

```
G1:  $FINISH \leftarrow 1, OVERFLOW \leftarrow 1$   
2:  $Y \leftarrow 0, C \leftarrow 0, OVERFLOW \leftarrow 0, i \leftarrow n$   
3:  $shl(CUV), shl(Y), i \leftarrow i - 1$   
(C + G)4:  $Y_0 \leftarrow 1, U \leftarrow U + X' + 1$   
Z'4: GOTO 3  
Z 4:  $FINISH \leftarrow 1$ 
```

Assignment:

Construct the flowchart yourself.

Shift-Subtract Division

- Algorithm

```
IF  $U \geq X$  THEN exit with overflow;  
 $Y = 0$ ;  $C = 0$ ;  
FOR  $i = 1$  TO  $n$  DO  
    {linear shift left  $CUV$ ;  
    linear shift left  $Y$ ;  
    IF  $CU \geq X$  THEN { $Y_0 = 1$ ,  $U = CU - X$ }}
```

- RTL code

```
G1:  $FINISH \leftarrow 1$ ,  $OVERFLOW \leftarrow 1$   
2:  $Y \leftarrow 0$ ,  $C \leftarrow 0$ ,  $OVERFLOW \leftarrow 0$ ,  $i \leftarrow n$   
3:  $shl(CUV)$ ,  $shl(Y)$ ,  $i \leftarrow i - 1$   
(C + G)4:  $Y_0 \leftarrow 1$ ,  $U \leftarrow U + X' + 1$   
Z'4: GOTO 3  
Z 4:  $FINISH \leftarrow 1$ 
```

RTL code to realize Shift-subtract Division Algorithm

G1: $FINISH \leftarrow 1$, $OVERFLOW \leftarrow 1$
2: $Y \leftarrow 0$, $C \leftarrow 0$, $OVERFLOW \leftarrow 0$, $i \leftarrow n$
3: $shl(CUV)$, $shl(Y)$, $i \leftarrow i - 1$
(C + G)4: $Y_0 \leftarrow 1$, $U \leftarrow U + X' + 1$
Z'4: GOTO **3**
Z 4: $FINISH \leftarrow 1$

IF $U \geq X$ THEN exit with overflow;
 $Y = 0$; $C = 0$;
 FOR $i = 1$ TO n DO
 {linear shift left CUV ;
 linear shift left Y ;
 IF $CU \geq X$ THEN $\{Y_0 = 1, U = CU - X\}$ }

Conditions	Micro-operations	i	C	U	V	Y	Z	FINISH
START		x	x	1001	0011	xxxx		0
1	NONE							
2	$Y \leftarrow 0$, $C \leftarrow 0$, $OVERFLOW \leftarrow 0$, $i \leftarrow 4$	4	0			0000	0	
3	$shl(CUV)$, $shl(Y)$, $i \leftarrow i - 1$	3	1	0010	0110	0000	0	
(C + G)4,Z'4	$Y_0 \leftarrow 1$, $U \leftarrow U + X' + 1$, GOTO 3			0101		0001		
3	$shl(CUV)$, $shl(Y)$, $i \leftarrow i - 1$	2	0	1010	1100	0010	0	
Z'4	GOTO 3							
3	$shl(CUV)$, $shl(Y)$, $i \leftarrow i - 1$	1	1	0101	1000	0100	0	
(C + G)4,Z'4	$Y_0 \leftarrow 1$, $U \leftarrow U + X' + 1$, GOTO 3			1000		0101		
3	$shl(CUV)$, $shl(Y)$, $i \leftarrow i - 1$	0	1	0001	0000	1010	1	
(C + G)4,Z4	$Y_0 \leftarrow 1$, $U \leftarrow U + X' + 1$, $FINISH \leftarrow 1$			0100		1011		1

$CU \geq X$
 8/12/2023

Arithmetic Unit

Remainder

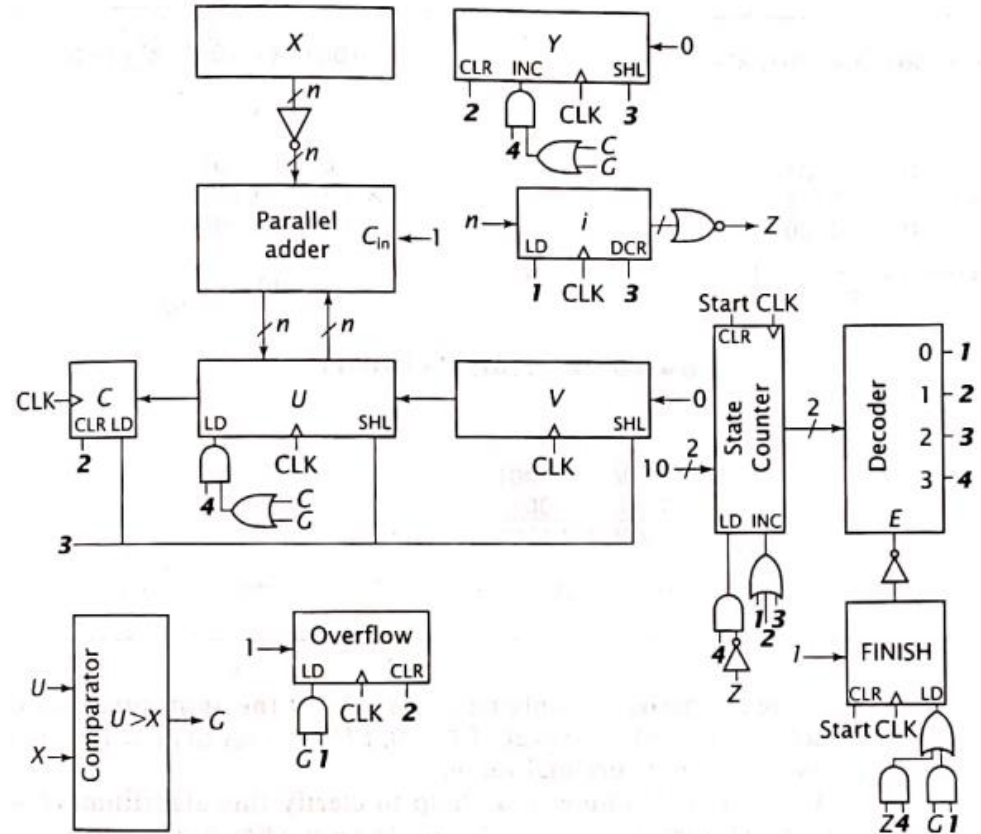
Quotient

Hardware Implementation of Shift-Subtract Division Algorithm

G1: $FINISH \leftarrow 1, OVERFLOW \leftarrow 1$
2: $Y \leftarrow 0, C \leftarrow 0, OVERFLOW \leftarrow 0, i \leftarrow n$
3: $shl(CUV), shl(Y), i \leftarrow i - 1$
(C + G)4: $Y_0 \leftarrow 1, U \leftarrow U + X' + 1$
Z'4: GOTO 3
Z 4: $FINISH \leftarrow 1$

```

IF U ≥ X THEN exit with overflow;
Y = 0; C = 0;
FOR i = 1 TO n DO
    {linear shift left CUV;
    linear shift left Y;
    IF CU ≥ X THEN {Y0 = 1, U = CU - X}}
    
```



Classwork:

a) Perform $14 \div 5$

6.7 Binary Coded Decimal

- It is the most popular format to represent decimal data
- In BCD, each 4 bits represent one decimal digit.

Decimal	Binary	BCD
0	0000 0000	0000 0000
1	0000 0001	0000 0001
2	0000 0010	0000 0010
3	0000 0011	0000 0011
4	0000 0100	0000 0100
5	0000 0101	0000 0101
6	0000 0110	0000 0110
7	0000 0111	0000 0111
8	0000 1000	0000 1000
9	0000 1001	0000 1001
10	0000 1010	0001 0000
11	0000 1011	0001 0001
12	0000 1100	0001 0010
13	0000 1101	0001 0011
14	0000 1110	0001 0100
15	0000 1111	0001 0101
16	0001 0000	0001 0110
17	0001 0001	0001 0111
18	0001 0010	0001 1000
19	0001 0011	0001 1001
20	0001 0100	0010 0000

BCD Addition

- Like other number system in BCD arithmetical operation may be required.
- BCD is a numerical code which has several rules for addition.
- The rules are given below in three steps with an example to make the idea of **BCD Addition** clear.

1. At first the given number are to be added using the rule of binary. For example,

Case 1:

$$\begin{array}{r} 1010 \\ + 0101 \\ \hline 1111 \end{array}$$

Case 2:

$$\begin{array}{r} 0001 \\ + 0101 \\ \hline 0110 \end{array}$$

BCD Addition

- In second step we have to judge the result of addition. Here two cases are shown to describe the rules of **BCD Addition**. In case 1 the result of addition of two binary number is **greater than 9**, which is not valid for BCD number. But the result of addition in case 2 is less than 9, which is valid for BCD numbers.
- If the four bit result of addition is greater than 9 and if a carry bit is present in the result then it is invalid and we have to add 6 whose binary equivalent is $(0110)_2$ to the result of addition. Then the resultant that we would get will be a valid binary coded number. In case 1 the result was $(1111)_2$, which is greater than 9 so we have to add 6 or $(0110)_2$ to it.

$$(1111)_2 + (0110)_2 = 0001\ 0101 = 15$$

Case 1:

$$\begin{array}{r} 1010 \\ + 0101 \\ \hline 1111 \end{array}$$

Case 2:

$$\begin{array}{r} 0001 \\ + 0101 \\ \hline 0110 \end{array}$$

BCD Addition

- As you can see the result is valid in BCD.
But in case 2 the result was already valid BCD, so there is no need to add 6. This is how BCD Addition could be.
- Now a question may arrive that why 6 is being added to the addition result in case BCD Addition instead of any other numbers. It is done to skip the six invalid states of binary coded decimal i.e from 10 to 15 and again return to the BCD codes.

BCD Addition

Example:1

Let, 0101 is added with 0110.

$$\begin{array}{r} 0101 \\ + 0110 \\ \hline 1011 \rightarrow \text{Invalid BCD number} \\ + 0110 \rightarrow \text{Add 6} \\ \hline 0001\ 0001 \rightarrow \text{Valid BCD number} \end{array}$$

Check your self.

$$(0101)_2 \rightarrow (5)_{10} \text{ and } (0110)_2 \rightarrow (6)_{10} \quad (5)_{10} + (6)_{10} = (11)_{10}$$

BCD Addition

Example:2

Now let 0001 0011 is added to 0010 0110.

$$\begin{array}{r} 0001\ 0001 \\ + 0010\ 0110 \\ \hline 0011\ 0111 \rightarrow \text{Valid BCD number} \end{array}$$

$$(0001\ 0001)_{BCD} \rightarrow (11)_{10}, (0010\ 0110)_{BCD} \rightarrow (26)_{10} \text{ and } (0011\ 0111)_{BCD} \rightarrow (37)_{10}$$
$$(11)_{10} + (26)_{10} = (37)_{10}$$

So no need to add 6 as because both

$$(0011)_2 = (3)_{10} \text{ and } (0111)_2 = (7)_{10}$$

are less than $(9)_{10}$. This is the process of BCD Addition.

BCD Adder Hardware Implementation

Invalid BCD numbers

1010

1011

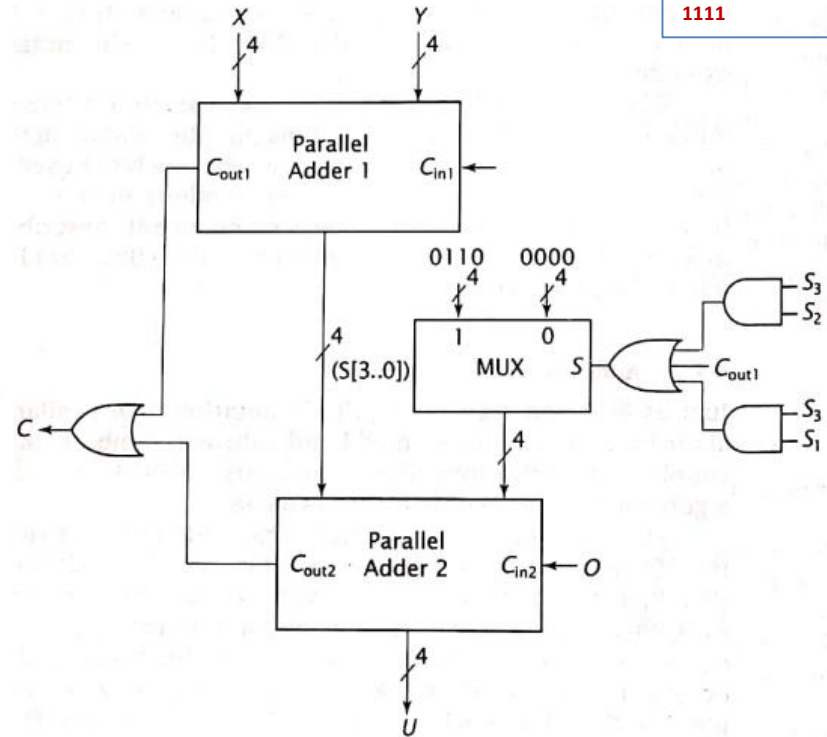
1100

1101

1110

1111

In this figure, notice that the two digits, X and Y , are first added together. Then this result is added to either 0 (0000) or 6 (0110) to produce the correct BCD sum. If the result, $S_3S_2S_1S_0$, is not a valid BCD digit, then either $S_3 \wedge S_2 = 1$ or $S_3 \wedge S_1 = 1$. (We can easily verify that the six invalid bit patterns 1010 through 1111 meet one or both of these conditions, and that the 10 valid BCD values 0000 through 1001 do not.) In this case, the multiplexer control bit is set to 1, which causes 6 to be added to the result, correcting its decimal representation. The multiplexer select bit is also set to 1 if the addition of X and Y generates a carry out, as when $X = 8$ and $Y = 9$. This also causes the circuit to add 6 to the original sum, again producing the correct value. If either parallel adder generates a carry out of 1, the carry out of the BCD adder should also be 1. Just as with binary adders, multiple copies of this hardware can be cascaded by connecting their carry out/carry in signals to form a multidigit BCD adder.



6.8 Specialized Arithmetic Hardware

- The different special arithmetic hardware are designed to speed up arithmetic computation.
- Coprocessors were formerly used to speed up computations.
- The coprocessor chip had specialized hardware that performed arithmetic calculations much more quickly than microprocessor.
- The coprocessor monitored instructions on system databus.
- When the microprocessor fetched an instruction that the coprocessor could execute, the coprocessor sent a signal to microprocessor indicating that it would perform computation.
- The coprocessor then calculates the result and send it to microprocessor.
- If the coprocessor was not present, the microprocessor executes the instruction in slow way.

Specialized Arithmetic Hardware

Three techniques to improve speed of arithmetic calculation:

1. Pipelining
2. Look up Tables
3. Wallace Tree multipliers

1. Pipelining

- An arithmetic pipeline is similar to an assembly line in a factory.
- Data enters a stage of the pipeline, which performs some arithmetic operation on the data.
- The results are then passed to the next stage, which performs its operations, and so on, until the final computation has been performed.
- Each stage performs only its specific function.
- It improves performance by overlapping computations; each stage can operate on different data simultaneously.

Pipelining example

- Pipelining is a technique of decomposing a sequential process into sub-operations, with each subprocess being executed in a special dedicated segment that operates concurrently with all other segments.
- The overlapping of computation is made possible by associating a register with each segment in the pipeline.
- The registers provide isolation between each segment so that each can operate on distinct data simultaneously.
- A clock is applied to all registers after enough time has elapsed to perform all segment activity.

Pipelining example

- Consider the below example: To perform the combined multiply and add operations with a stream of numbers $A_i * B_i + C_i$ for $i = 1, 2, 3, \dots, 7$
- Each sub-operation is to be implemented in a segment within a pipeline.

$R1 \leftarrow A_i, R2 \leftarrow B_i$

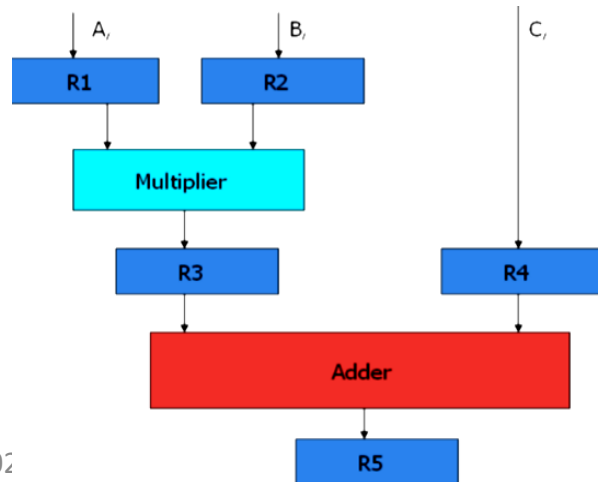
Input A_i and B_i

$R3 \leftarrow R1 * R2, R4 \leftarrow C_i$

Multiply and input C_i

$R5 \leftarrow R3 + R4$

Add C_i to product



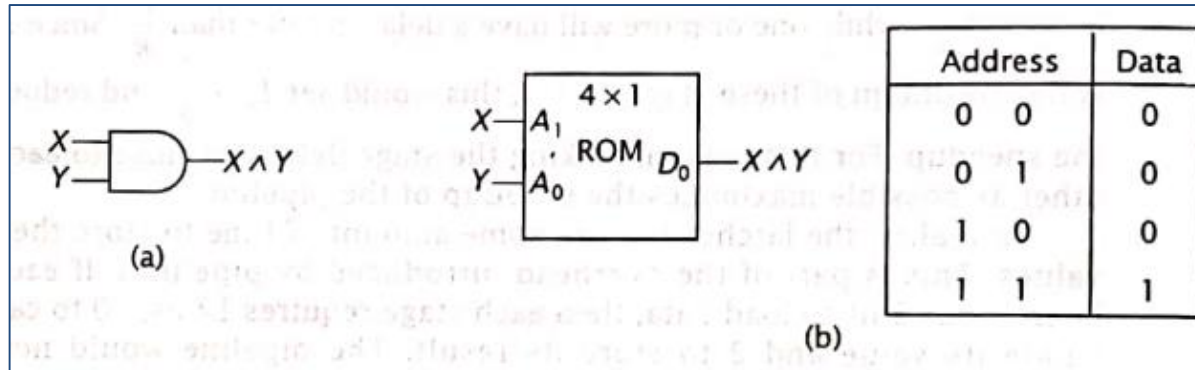
Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A_1	B_1	--	--	--
2	A_2	B_2	$A_1 * B_1$	C_1	--
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	--	--	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	--	--	--	--	$A_7 * B_7 + C_7$

2. Lookup Table

- A **combinational circuit** can be implemented by a ROM configured as **LookUp table**.
- The inputs to the combinational circuit serve as address inputs of the ROM.
- The data outputs of ROM corresponds to the outputs of combinational circuit.
- The **ROM is programmed with data** such that the correct values are output for any possible input values.

2. Lookup Table

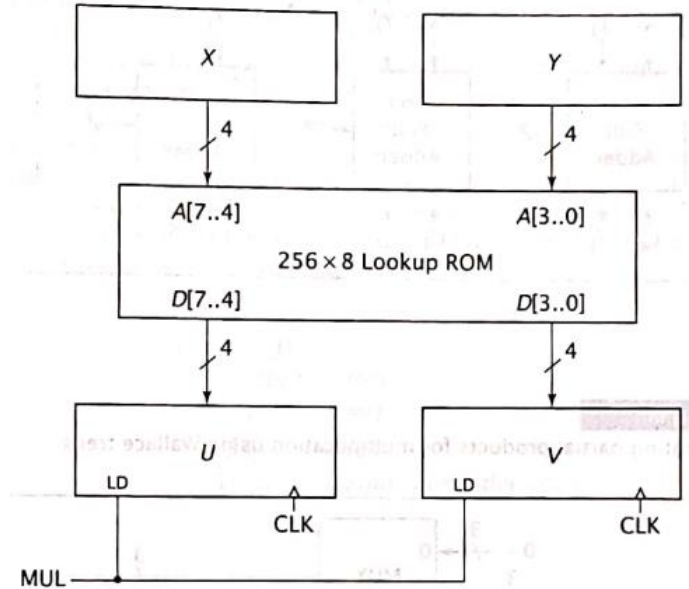
- Example: Let us consider **4 x 1 ROM** programmed to mimic **a two input AND gate**.
- The inputs to the AND gate serve as the address inputs to the ROM, and the output of the ROM corresponds to the AND gate output.
- By programming the ROM with the data shown, it outputs the same values as the AND gate for all possible of X and Y.



(a) An AND gate and (b) its lookup ROM equivalent

A Multiplier implemented using a Lookup ROM

- Recalling the shift-add multiplication algorithm for unsigned numbers, we can construct a hardware implementation for a multiplier using Lookup table.
- $UV \leftarrow X.Y$
- Where U, V, X and Y are 4-bit registers.
- Registers X and Y supply the address inputs to the lookup ROM. Its inputs are the product of X and Y, and are routed to registers U and V.
- Each of the 256 locations must contain the 8-bit product of X and Y.
- For example, location 1011 1101 contains the data 1000 1111 (i.e 143), the product of 11 and 13.



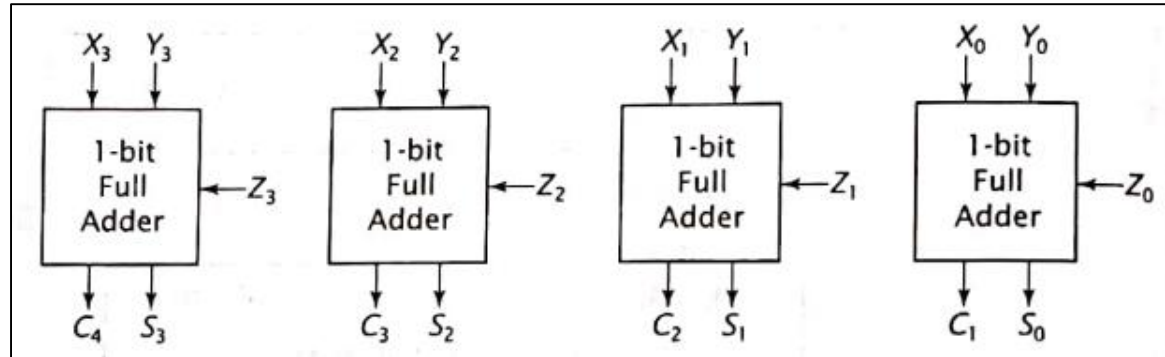
Learn more from: <https://hardwarebee.com/overview-of-lookup-tables-in-fpga-design/>

Classwork

- Design a Lookup ROM to implement the function $f=xy + y'z$.

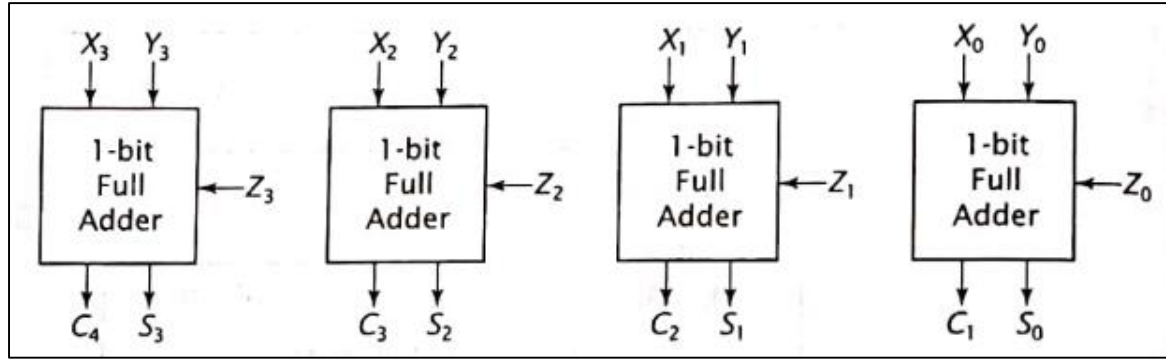
3. Wallace Tree

- A Wallace tree is a **combinational circuit used to multiply two numbers**. It requires more hardware but can produce a product in less time.
- It requires carry save adders and only one parallel adder.
- A carry-save adder can add 3 values simultaneously instead of just two. It outputs both sum and a set of carry bits.
- This is illustrated below:



A carry-save adder

Wallace Tree



A carry-save adder

- Each bit S_i is the binary sum of bits X_i , Y_i and Z_i
- Carry bit C_{i+1} is the carry generated by this sum.
- To form a final sum, C and S must be added together.
- Because carry bits do not propagate through the adder, it is faster than a parallel adder.

An example of addition for three numbers : 7+11+2

	c_3	c_2	c_1	c_0
C	0	0	1	1
X	0	1	1	1
Y	1	0	1	1
Z	0	0	1	0
S	1	1	1	0

Adding S and C using a parallel adder, it produces the result 10100 (20) which is by $00110 + 1110 = 10100$.

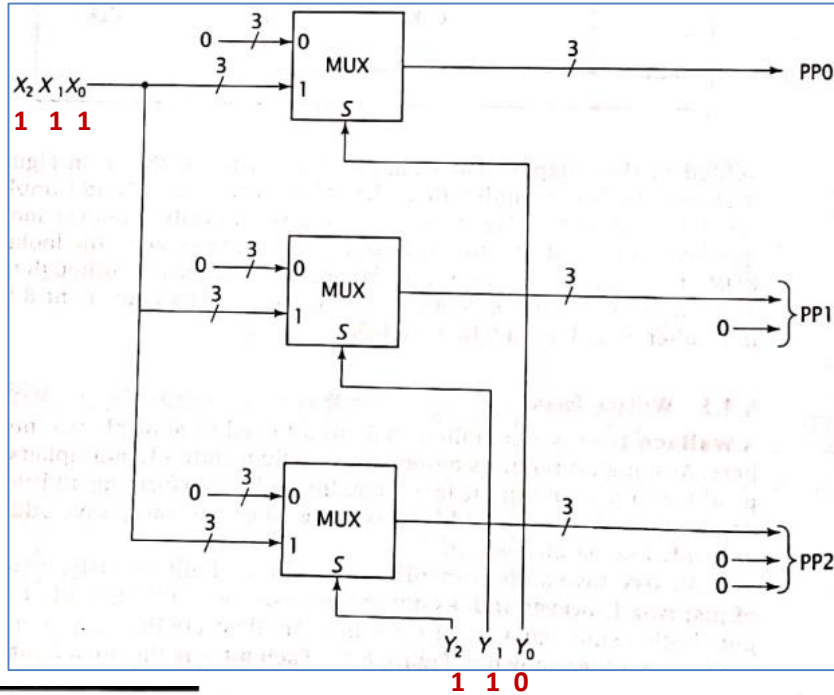
Wallace Tree

- To use a carry-save adder to perform multiplication we first calculate the partial products (PP) of the multiplication, then input them to the carry-save adder.
- Consider a numeric example multiplication of 7x6.

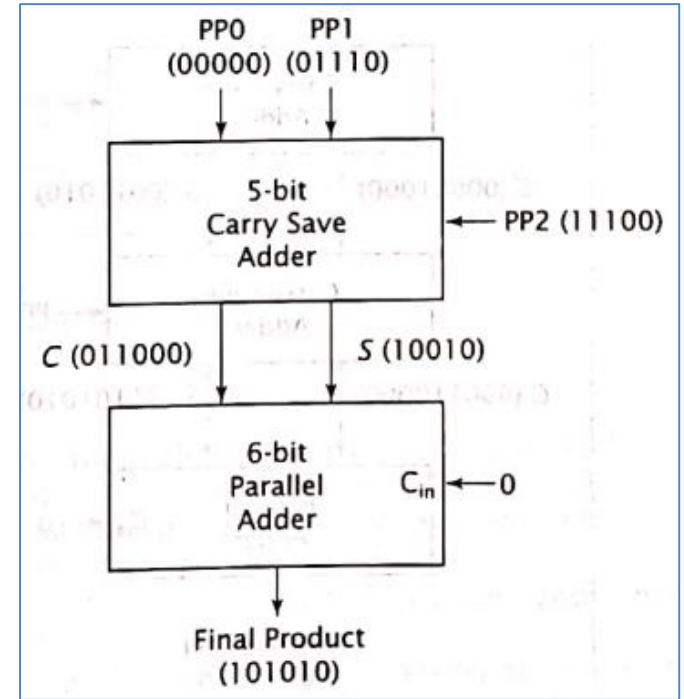
$$\begin{array}{rcl} x = & 111 & \\ y = & \underline{110} & \\ & 000 & \leftarrow PP0 \\ & 111 & \leftarrow PP1 \\ & \underline{111} & \leftarrow PP2 \\ & 101010 & \leftarrow \text{Final sum calculated} \end{array}$$

A 3x3 Wallace Tree Multiplier

$$\begin{array}{r}
 x = 111 \\
 y = 110 \\
 \hline
 000 \leftarrow PPO \\
 111 \leftarrow PPI \\
 \underline{111} \leftarrow PP2 \\
 101010 \leftarrow \text{Final sum calculated}
 \end{array}$$



Generating partial products for multiplication using Wallace trees



A 3×3 multiplier constructed using a carry-save adder

6.9 Floating Point Numbers

- The floating point representation of a number has two parts.
 - The **first part** represent a signed fixed point number called **mantissa**.
 - The **second part** designates the position of decimal (or binary) point and is called **exponent**.
- The fixed point mantissa may be fraction or an integer.
- For example:
 - The decimal number **6132.789** is represented in floating point as 0.6132789×10^4
- This is in the form: $m \times r^e$
 - where
 - **m** is mantissa and
 - **e** is exponent.

Fraction	Exponent
0.6132789	04

- The value of exponent indicates that actual position of decimal point is four position to right of indicate decimal point in fraction

6.9 Floating Point Numbers

- For example a decimal number $3.25_{10} = 11.01_2$
 $= 1.101_2 \times 10_2^1$
 $= 1.101 \times 2^1$

Fraction	Exponent
1.101	01

6.10 IEEE 754 floating Point Standard, Numeric Format

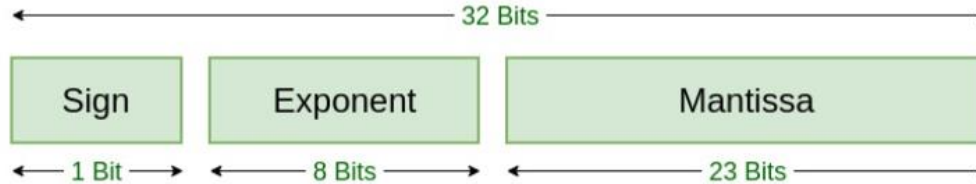
- The IEEE Standard for Floating-Point Arithmetic (IEEE 754) is a technical standard for floating-point computation which was established in 1985 by the **Institute of Electrical and Electronics Engineers (IEEE)**.
- The standard addressed many problems found in the diverse floating point implementations that made them difficult to use reliably and reduced their portability.
- **IEEE Standard 754** floating point is **the most common** representation today for real numbers on computers, including Intel-based PC's, Macs, and most Unix platforms.
- There are several ways to represent floating point number but **IEEE 754 is the most efficient** in most cases.

6.10 IEEE 754 floating Point Standard, Numeric Format

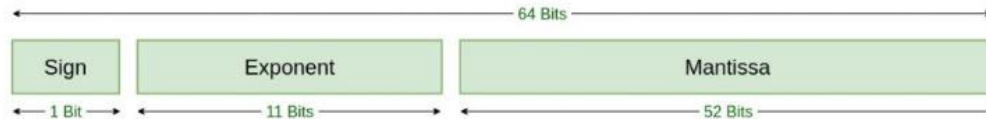
- There are several ways to represent floating point number but IEEE 754 is the most efficient in most cases.
- IEEE 754 has 3 basic components:
 - **The Sign of Mantissa –**
This is as simple as the name. 0 represents a positive number while 1 represents a negative number.
 - **The Biased exponent –**
The exponent field needs to represent both positive and negative exponents. A bias is added to the actual exponent in order to get the stored exponent.
 - **The Normalised Mantisa –**
The mantissa is part of a number in scientific notation or a floating-point number, consisting of its significant digits. Here we have only 2 digits, i.e. 0 and 1. So a normalised mantissa is one with only one 1 to the left of the decimal.

6.10 IEEE 754 floating Point Standard, Numeric Format

- IEEE 754 numbers are divided into two based on the above three components: single precision and double precision.



Single Precision
IEEE 754 Floating-Point Standard



Double Precision
IEEE 754 Floating-Point Standard

Denormalization

- Denormalize: 1.101×2^1
- Here: exponent of base 2 =1
 Bias=127
 $E=127+1 =128$

Hence the denormalized encoding of 1.101×2^1 is:

Sign	Exponent	Significant no (fractional no)
0	1000 0000	101 0000 0000 0000 0000 0000
<i>1 bit</i>	<i>8 bits</i>	<i>23 bits</i>

Reference: https://www.youtube.com/watch?v=T_Wxrp-2DZQ

Represent $(1259.125)_{10}$ in single precision format

- Step 1: Convert decimal number to binary number
$$1259.125_{10} = 10011101011.001_2$$
- Step 2: Normalize the number in the form: $(1.N)_2 \times 2^E$
- $10011101011.001_2 = 1.0011101011001 \times 2^{10}$
- Here: exponent of base 2 = 10

Bias=127

$E=127+10=137=10001001$

Hence the denormalized encoding of $1.0011101011001 \times 2^{10001001}$ is:

Sign	Exponent	Significant no (fractional no)
0	1000 1001	001 1101 0110 0100 0000 0000
1 bit	8 bits	23 bits

Reference: https://www.youtube.com/watch?v=T_Wxrp-2DZQ

Normalization

Sign	Exponent	Significant no (fractional no)
1	1000 0000	101 0000 0000 0000 0000 0000
<i>1 bit</i>	<i>8 bits</i>	<i>23 bits</i>

Here, sign bit =1 Hence, the no. is negative

Exponent =1000 0000

So, $E = 128 - 127 = 1$

Also, $S = 101\ 0000\ 0000\ 0000\ 0000\ 0000$

$= 1.101$

Therefore, binary no is: $= -1.101 \times 2^1$

$= -1.101 \times 10$

$= -11.01_2$

Floating point Addition/Subtraction

- Let

$$X = \pm X_S + B^{\pm X_E}$$

$$Y = \pm Y_S + B^{\pm Y_E}$$

- For $X_E = Y_E$,

Addition	$X+Y = (X_S + Y_S) \times B^{X_E}$
Subtraction	$X-Y = (X_S - Y_S) \times B^{X_E}$
Multiplication	$X*Y = (X_S * Y_S) \times B^{X_E + Y_E}$
Division	$X \div Y = (X_S \div Y_S) \times B^{X_E - Y_E}$

Given two fractional no: $X = 0.3 \times 10^2$ $Y = 3.0 \times 10^2$ Perform addition.

Solution:

$$\begin{array}{lll} \text{Here,} & X_S = 0.3 & B^{X^E} = 10^2 \\ & Y_S = 3.0 & B^{Y^E} = 10^2 \end{array}$$

Now,

$$\begin{aligned} X+Y &= (X_S + Y_S) \times B^{X^E} \\ &= (0.3 + 3.0) \times 10^2 \\ &= 3.3 \times 10^2 \end{aligned}$$

Perform addition for the given binary floating numbers. And also normalize the result.

$$X = -1.01 \times 2^2$$

$$Y = 1.1 \times 2^4$$

Solution:

$$\begin{aligned} X &= -1.01 \times 2^2 \\ &= -0.0101 \times 2^4 \end{aligned}$$

$$\begin{aligned} Y &= 1.1 \times 2^4 \\ &= 1.1 \times 2^4 \end{aligned}$$

Here,

$$\begin{aligned} X+Y &= (X_S + Y_S) \times B^{X_E} \\ &= (-0.0101 + 1.1) \times 2^4 \\ &= 1.0011 \times 2^4 \end{aligned}$$

Now to normalize for 1.0011×2^4

Exponent of base 2 = 4

Bias = 127

E = 127 + 4 = 131

for 8 bit

Hence, normalized encoding of 1.0011×2^4 is:

0	1000 1000	0011 0000 0000 0000 0000 0000
----------	------------------	--------------------------------------

Assignment:

1. Trace the multiplication of -9 and -12 using RTL code of Booth's algorithm. Also verify the obtained product.
2. Write RTL code for Booth's algorithm. Trace the multiplication of 7 and -4 using RTL code of Booth's algorithm. Also verify the obtained product.
3. Show the Wallace tree to perform 4x4 multiplication for 1101 x 1110.
4. Describe about BCD addition.
5. What is Lookup ROM? Describe in brief demonstrating a Lookup ROM working as XOR gate.
6. Show the memory content of a Lookup ROM equivalent to input OR gate.
7. Design a Lookup ROM to implement the function $f = xy + y + z$.
8. Design a Lookup ROM to implement the function $f = xyz + yz$.

Exam Questions:

1. Trace the multiplication of (4) and (3) using RTL code of shift Add Multiplication Algorithm. [PU 2017 Fall]
2. Write the RTL code for Booth's Algorithm. Show the hardware implementation for the RTL code. [PU 2017 Fall]
3. What is Wallace Tree? Describe in brief demonstrating a 3x3 multiplier being constructed using Wallace tree and carry save adder. [PU 2017 Fall]
4. What is Lookup ROM? Show the memory content of a Lookup ROM equivalent to input OR gate. [PU 2017 Spring]
5. Write the RTL code for Shift Add Algorithm. Show the hardware implementation for the RTL code. [PU 2017 Spring]
6. What is Arithmetic Pipelining? Describe in brief with an example. [PU 2017 Spring]
7. List out the RTL code for arithmetic and logical operations. Also show the implantation of addition and subtraction using parallel adder. [PU 2018 Fall]
8. Write down the RTL code for Booth's Algorithm. [PU 2018 Fall]
9. What is Lookup ROM? Describe in brief demonstrating a Lookup ROM working as XOR gate. [PU 2018 Fall]
10. Trace RTL code of shift add multiplication algorithm for multiplication of 3 and 4. [PU 2018 Spring]

Exam Questions:

11. What is Lookup ROM? Illustrate with an example how it works? [PU 2018 Spring]
12. Trace RTL code for Booth's Algorithm for multiplication of 9 and -3. [PU 2019 Spring]
13. What is Lookup ROM? Design a Lookup ROM to implement the function $f=xy + y'z$. [PU 2019 Spring]
14. Perform -7×-2 using booth's algorithm. Can booth's algorithm be used if both numbers are positive, if yes how? [PU 2020 Spring]
15. Write short notes on:
 - a) Signed and Unsigned Number representation. [PU 2017 Spring]
 - b) BCD addition [PU 2018 Fall]
 - c) BCD adder [PU 2018 Spring]

End of Chapter